

# Image Recognition to Identify Species of Flowers

## Abstract

This project demonstrates the building of a Convolutional Neural Network (CNN) model for classifying images of flowers into five species using the *tf\_flowers* dataset. The dataset contained 3,670 labelled images, and the models were trained from scratch using TensorFlow. The CNN architecture was tested, together with regularization strategies such as dropout, batch normalization, and data augmentation to reduce overfitting. Additionally, hyperparameters which include kernel size, learning rate, stride, batch size, and optimizer were tuned. Training and validation metrics were closely aligned, which indicates that the best-performing model had little overfitting and strong validation accuracy. Results were evaluated using validation accuracy, validation loss curves, classification report (precision, recall, F1 score) and confusion matrix. This work shows how well CNN performs image recognition tasks and it also highlights how design and training strategies affect model performance.

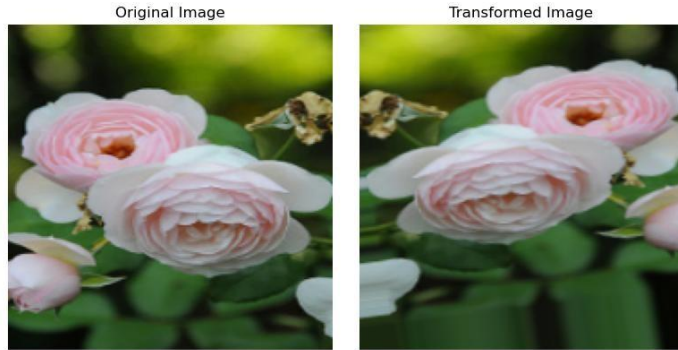
## 1. Introduction

Image classification using deep learning has become more successful, especially when Convolutional Neural Networks (CNNs) are used. CNNs are ideal for tasks like flower species identification because of their ability to identify patterns and features in photos. This project explores the use of CNNs to classify images from the *tf\_flowers* dataset, which contains five types of flowers including daisy, dandelion, rose, sunflower, and tulip. The aim of this work was to build and evaluate a CNN model that is capable of classifying these images accurately, as well as exploring the effects of regularization and hyperparameter tuning on model performance.

## 2. Methodology

### 2.1 Dataset and Preprocessing

The dataset, which includes 3,670 labeled photos of five distinct flower species, was obtained from TensorFlow's official repository (*tf\_flowers*). TensorFlow's image dataset was used to load the images from the local directory into the training, validation, and test sets. To standardize the inputs, every image was resized to 180 x 180 pixels and rescaled to a range of 0 and 1. To reduce overfitting and artificially boost dataset diversity, data augmentation was used during training (Shorten and Khoshgoftaar, 2019). This method improved the model's generalization by exposing it to slightly different versions of the same class during the training. Figure 2.1 shows an image of the dataset, before and after augmentation.



*Fig 2.1: Image, before and after augmentation*

## **2.2 Model Architecture**

The model was implemented using a custom function that builds CNN model. The function takes different hyperparameters for flexibility in tuning. It is reused with different optimizers and configurations.

### **2.2.1 Convolutional Layers:**

The model's architecture is made up of three 2-dimensional (2D) convolutional layers, followed by a pooling layer and batch normalization (optional). The first convolution layer used 32 filters, second layer used 64 filters, and third layer used 128 filters. The flattening layer was used after the final convolutional layer to convert the 3D feature maps into a 1D vector so that the output can be fed into the dense layers for final classification. Fully connected dense layer with 64 units was used. A final output layer with 5 units and softmax activation function was used for each flower class.

### **2.2.2 Activation Functions:**

ReLU activation function was used throughout the hidden layers to add non-linear behavior, while Softmax function was used in the final output layer for the five classes.

### **2.2.3 Regularisation Methods:**

A dropout rate of 0.5 was used after the dense layer to minimize overfitting by randomly dropping neuron during training (Srivastava et al., 2014).

Batch normalization was applied optionally after each of the convolutional layers to stabilize learning and speed up convergence (Ioffe and Szegedy, 2015).

### **2.2.4 Justification of Design:**

Best practices in VGG-style architectures, which effectively stack the filters to approximate larger receptive fields while lowering the parameter count, are in line with the use of small kernel size 3 by 3 filters with stride of 1 (Alzubaidi et al., 2021). Exploring which pooling type was most effective for the flower classification was made easier by the ability to switch between maximum and average pooling. Batch normalization and dropout provided complementary regularization techniques that improved generalization and reduced overfitting.

#### 2.2.5 Hyperparameter Tuning:

Different hyperparameters were used to improve the model's performance during training. These included the learning rate, batch size, optimizer, kernel size, stride length, pooling type and batch normalization. Learning rates such as 0.01, 0.001 and 0.0001 were tested. Batch sizes of 32, 64, and 128 were compared in the training. Adam and SGD optimizers were used. These tunings helped to identify the best configuration for training an accurate and generalized model.

#### 2.2.6 Evaluation Metrics:

Model performance was measured using training and validation accuracy/loss curves in the plots, classification report (precision, recall, F1-score) and confusion matrix.

### 3. Results

The outcomes of training and assessing various CNN models on the `tf_flowers` dataset are displayed in this section. The effectiveness of the model was assessed using performance metrics such as confusion matrices and training and validation accuracy/loss.

The baseline model (the default parameters used before tuning) trained with a learning rate of 0.01, performed poorly with a validation accuracy of 26%. However, the learning rate was reduced by 10% (0.001), and the model performed well with a validation accuracy of 68%. The different models were trained at a dropout rate of 0.5.

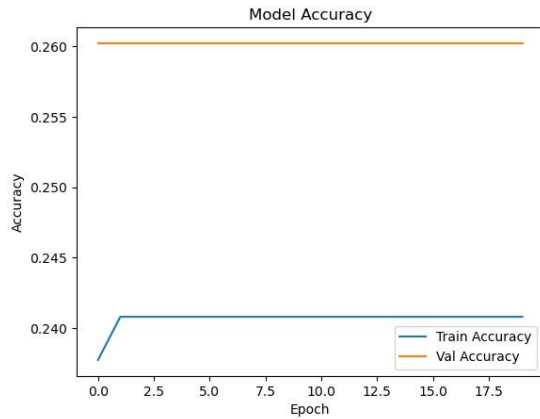


Fig 3.1: Accuracy plot for baseline model

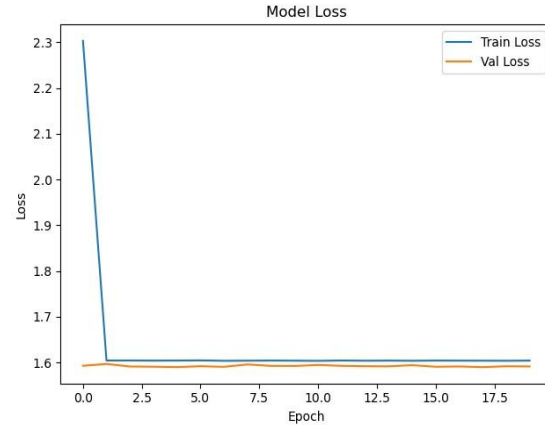


Fig 3.2: Loss plot for baseline model

### 3.1 Hyperparameter Tuning:

Table 3.1 summarizes different hyperparameter tuning and their corresponding validation accuracies. Models that used a learning rate of **0.001**, a batch size of **32**, and included dropout and batch normalization performed better overall. Models trained with high learning rate (0.01) showed signs of underfitting and stagnant. All models were trained with a dropout rate of 0.5.

Table 3.1: CNN Model configurations

| Learning Rate | Batch Size | Kernel Size | Stride | Batch Norm | Pooling | Val Acc (%) | Val Loss |
|---------------|------------|-------------|--------|------------|---------|-------------|----------|
| 0.01          | 32         | 3x3         | 1x1    | False      | Max     | 0.2602      | 1.5912   |
| 0.001         | 32         | 3x3         | 1x1    | False      | Max     | 0.6826      | 1.8552   |
|               | 64         | 3x3         | 1x1    | False      | Max     | 0.6757      | 1.5331   |
|               | 128        | 3x3         | 1x1    | False      | Max     | 0.6757      | 1.4533   |
|               | 128        | 3x3         | 2x2    | False      | Max     | 0.7248      | 0.7740   |
|               | 128        | 5x5         | 2x2    | False      | Max     | 0.7289      | 0.8596   |
|               | 128        | 5x5         | 2x2    | False      | Avg     | 0.6717      | 0.8780   |
|               | 32         | 3x3         | 1x1    | False      | Avg     | 0.6880      | 1.4944   |
|               | 32         | 3x3         | 1x1    | True       | Avg     | 0.3488      | 2.0364   |
|               | 128        | 5x5         | 2x2    | True       | Avg     | 0.6458      | 1.0349   |
| 0.0001        | 32         | 3x3         | 1x1    | True       | Max     | 0.6975      | 1.1980   |
|               | 128        | 5x5         | 2x2    | False      | Max     | 0.6185      | 0.9783   |

|  |    |     |     |       |     |        |        |
|--|----|-----|-----|-------|-----|--------|--------|
|  | 32 | 3x3 | 1x1 | False | Max | 0.6662 | 0.9152 |
|--|----|-----|-----|-------|-----|--------|--------|

According to Table 3.1, model improved consistently by the learning rate of 0.001, larger batch sizes (128), the use of maximum pooling with a kernel size of 5 by 5 and stride of 2 by 2. The best result which has a validation accuracy of 73%, was achieved without batch normalization. Batch normalization on the other hand, improved performances in some cases and other times degraded it. Overall, larger kernels and strides helped to extract richer features, while MaxPooling performed better than AvgPooling in most settings.

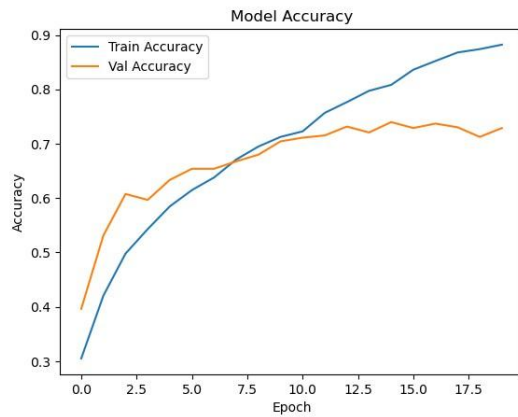


Fig 3.3: Model accuracy of final model

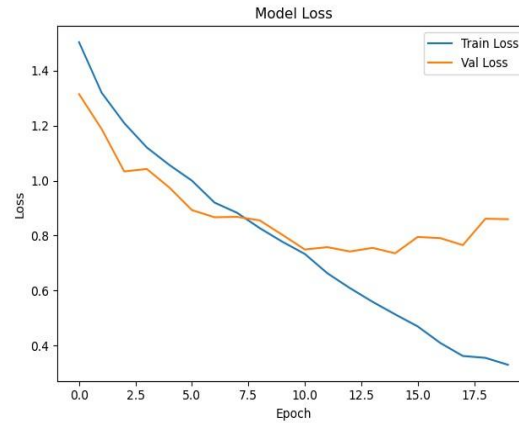


Fig 3.4: Model loss of final model

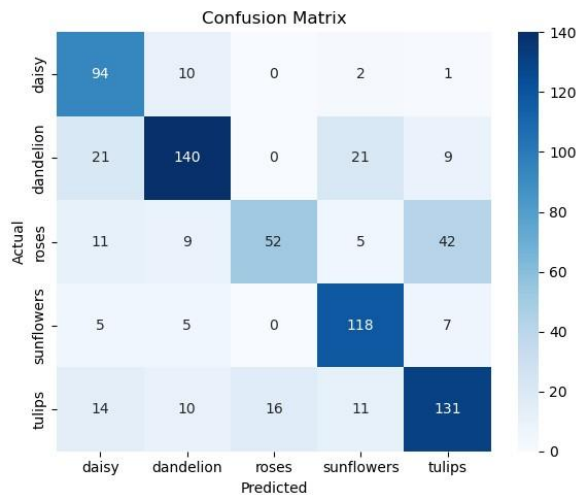


Fig 3.5: Confusion matrix of final model

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| daisy        | 0.65      | 0.88   | 0.75     | 107     |
| dandelion    | 0.80      | 0.73   | 0.77     | 191     |
| roses        | 0.76      | 0.44   | 0.56     | 119     |
| sunflowers   | 0.75      | 0.87   | 0.81     | 135     |
| tulips       | 0.69      | 0.72   | 0.70     | 182     |
| accuracy     |           |        | 0.73     | 734     |
| macro avg    | 0.73      | 0.73   | 0.72     | 734     |
| weighted avg | 0.74      | 0.73   | 0.72     | 734     |

Fig 3.6: Classification Report of final model

The best performed model in the results above (trained with Adam optimizer) which had a validation accuracy of 73% was compared with SGD optimizer. Adam significantly outperformed SGD which achieved a validation accuracy of 28% (See Table 3.2). This result relates with findings by Kingma and Ba (2014), who demonstrated that Adam's adaptive learning rate and momentum help it converge faster and more efficiently than other traditional optimizers like SGD.

Table 3.2: Comparison of Adam and SGD optimizers

| Optimizer | Val Accuracy (%) | Val Loss (%) |
|-----------|------------------|--------------|
| Adam      | 0.7289           | 0.8596       |
| SDG       | 0.2793           | 1.5823       |

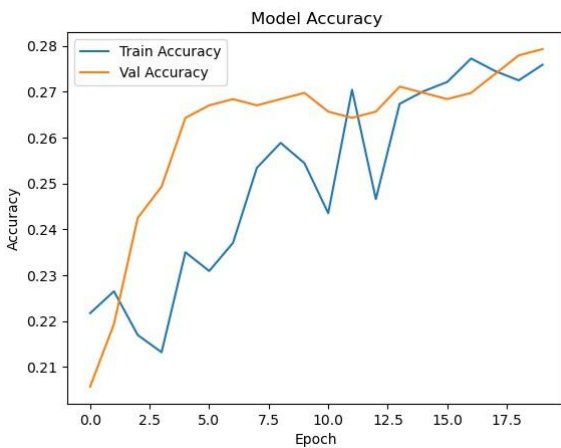


Fig 3.7: Model accuracy for SGD optimizer

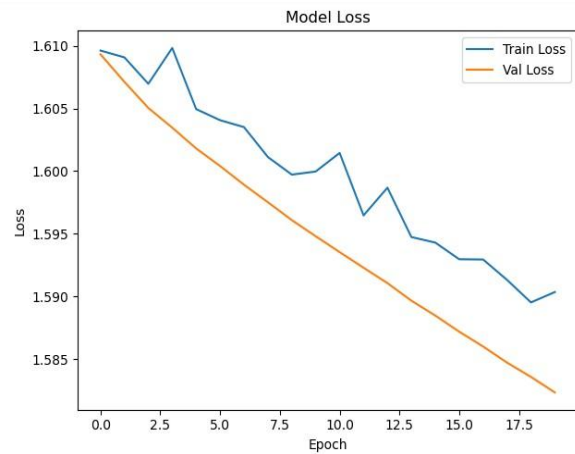


Fig 3.8: Model loss for SGD optimizer

## 4. Discussion

The model performed best with a 0.001 learning rate, 5 by 5 kernels, 2 by 2 stride, and batch size of 128, achieving a validation accuracy of 73%. Maximum pooling continuously performed better than the average pooling, and batch normalization showed mixed results by performing well in some cases and not performing well in others.

## **4.1 Model Architecture**

All trained models used three convolutional layers with filter sizes (64, 64, and 128), followed by a dense layer with 64 units. This architecture allowed the model to learn more complicated features from the images, thereby increasing the accuracy and reducing overfitting. The ReLU activation function was used after each layer to help the model learn non-linear patterns effectively.

## **4.2 Regularisation Methods**

The two key regularisation methods used were dropout and batch normalization. Dropout rate of 0.5 was added after the dense layer to help minimize overfitting by randomly removing neurons during training of the model (Srivastava et al., 2014). Added batch normalization after the hidden layers in some models to stabilise training and improve performance. These methods helped in bridging the accuracy gap between training and validation.

Data augmentation (such as random flips, rotation, and zoom) was also important. It increased the variety of training images, helping the model to improve and generalise to new data.

## **4.3 Hyperparameter Tuning**

Several hyperparameters were tested, including learning rate, dropout rate, batch size, pooling type, stride, kernel size and optimizer type. The learning rate had the biggest effect on the model. A learning rate value of 0.001 gave the best results. 0.01 which is a higher rate, caused an unstable training, while the lowest rate (0.0001) made the training very slow and outperformed 0.01 rate.

Batch size also had significant impact on the results. Smaller batch sizes (32) slightly improved validation accuracy but made the training slow. Comparison between optimizers (Adam and SGD) showed that Adam significantly outperformed SGD, which confirms that adaptive optimizers are more efficient for CNN model training (Kingma and Ba, 2014).

## **4.4 Evidence of Overfitting**

Overfitting was noticed in some models, especially those that were trained without batch normalization. This was demonstrated by increasing validation loss after a few epochs and training accuracy being significantly higher than validation accuracy. Figure 4.1 shows that there is evidence of overfitting in that model. The validation accuracy plateaued after the initial training stage while the training accuracy continued to increase above 90%, thereby creating a wide performance gap. This shows that the model learned the training data too well but is then struggling to generalize to new data.

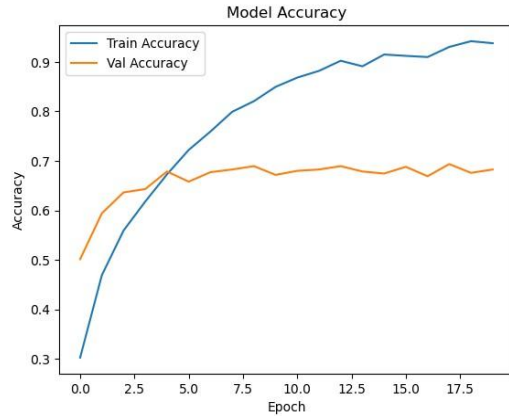


Fig 4.1: Model accuracy showing signs of overfitting

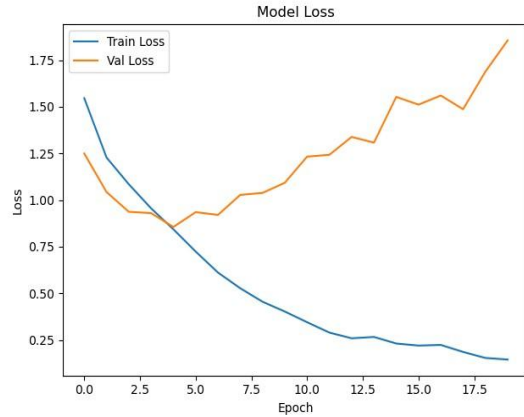


Fig 4.2: Model loss showing signs of overfitting

## 5. Conclusion

This project successfully demonstrated the use of Convolutional Neural Networks (CNNs) for classifying flower species using the *tf\_flowers* dataset. The model's performance was enhanced greatly by experimenting with different regularization methods and hyperparameter tunings. The model that performed best achieved a strong validation accuracy with little overfitting and this was achieved by using dropout, batch normalization and data augmentation. Overall, the project highlights how important it is to tune hyperparameter when training CNN models and how effective CNN models are in image classification tasks.



## References

Hu, J., 2025. Flower Pictures Recognition Based on the Advanced Convolutional Neural Network with Oxford Flowers 102 Dataset.

Shorten, C. and Khoshgoftaar, T.M., 2019. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), p.60.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1), pp.1929-1958.

Ioffe, S. and Szegedy, C., 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research*, 37, pp.448–456.

Alzubaidi, L., Zhang, J., Humaidi, A.J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M.A., Al-Amidie, M. and Farhan, L., 2021. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of big Data*, 8(1), p.53.

Kingma, D.P. and Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.